Chapter 22. Organizing Your Application

Throughout this book we have described various components of an application built on top of Kubernetes. We have described how to wrap programs up as containers, place those containers in Pods, replicate those Pods with ReplicaSets, and roll them out with Deployments. We have even described how to deploy stateful and real-world applications that collect these objects into a single distributed system. But we have not covered how to actually work with such an application in a practical way. How can you lay out, share, manage, and update the various configurations that make up your application? That is the topic of this chapter.

Principles to Guide Us

Before digging into the concrete details of how to structure your application, it's worth considering the goals that drive this structure. Obviously, reliability and agility are the general goals of developing a cloud native application in Kubernetes, but how does this relate to how you design your application's maintenance and deployment? The following sections describe three principles that can guide you in designing a structure that best suits these goals. The principles are:

- Treat filesystems as the source of truth
- Conduct code review to ensure the quality of changes
- Use feature flags to stage rollouts and rollbacks

Filesystems as the Source of Truth

When you first begin to explore Kubernetes, as we did in the beginning of this book, you generally interact with it imperatively. You run commands like `kubectl run` or `kubectl edit` to create and modify Pods or other objects running in your cluster. Even when we started exploring how to write and use YAML files, this was presented in an ad-hoc manner, as if the file itself is just a way station on the road to modifying the state of the cluster. In reality, in a true productionized application the opposite should be true.

Rather than viewing the state of the cluster—the data in `etcd`—as the source of truth, it is optimal to view the filesystem of YAML objects as the source of truth for your application. The API objects deployed into your Kubernetes cluster(s) are then a reflection of the truth stored in the filesystem.

There are numerous reasons why this is the right point of view. The first and foremost is that it largely enables you to treat your cluster as if it is immutable infrastructure. As we have moved into cloud native architectures, we have become increasingly comfortable with the notion that our applications and their containers are immutable infrastructure, but treating a cluster as such is less common. And yet, the same reasons for moving our applications to immutable infrastructure apply to our clusters. If your cluster is a snowflake you made by applying random YAML files downloaded from the internet ad hoc, it is as dangerous as a virtual machine built from imperative bash scripts.

Additionally, managing the cluster state via the filesystem makes it very easy to collaborate with multiple team members. Source-control systems are well understood and can easily enable multiple people to edit the state of the cluster simultaneously, while making conflicts (and the resolution of those conflicts) clear to everyone.

NOTE

It is absolutely a first principle that *all applications deployed to Kubernetes should first be described in files stored in a filesystem*. The actual API objects are then just a projection of this filesystem into a particular cluster.

The Role of Code Review

It wasn't long ago that code review for application source code was a novel idea. But it is clear now that multiple people looking at a piece of code before it is committed to an application is a best practice for producing high-quality, reliable code.

It is therefore surprising that the same is somewhat less true for the configurations used to deploy those applications. All of the same reasons for reviewing code apply directly to application configurations. But when you think about it, it is also obvious that code review of these configurations is critical to the reliable deployment of services. In our experience, most service outages are self-inflicted via unexpected consequences, typos, or other simple mistakes. Ensuring that at least two people look at any configuration change significantly decreases the probability of such errors.

NOTE

The second principle of our application layout is that it must facilitate the review of every change merged into the set of files that represents the source of truth for our cluster.

Feature Gates

Once your application source code and your deployment configuration files are in source control, one of the most common questions is how these repositories relate to one another. Should you use the same repository for application source code and configuration? This can work for small projects, but in larger projects it often makes sense to separate the two. Even if the same people are responsible for both building and deploying the application, the perspectives of the builder versus those of the deployer are different enough that this separation of concerns makes sense.

If that is the case, then how do you bridge the development of new features in source control with the deployment of those features into a production environment? This is where feature gates play an important role.

The idea is that when some new feature is developed, that development takes place entirely behind a feature flag or *gate*. This gate looks something like:

```
if (featureFlags.myFlag) {
   // Feature implementation goes here
}
```

There are a variety of benefits to this approach. First, it lets the team commit to the production branch long before the feature is ready to ship. This enables feature development to stay much more closely aligned with the `HEAD` of a repository, and thus you avoid the horrendous merge conflicts of a long-lived branch.

Working behind a feature flag also means that enabling a feature simply involves making a configuration change to activate the flag. This makes it very clear what changed in the production environment, and very simple to roll back the feature activation if it causes problems.

Using feature flags thus both simplifies debugging and ensures that disabling a feature doesn't require a binary rollback to an older version of the code that would remove all of the bug fixes and other improvements made by the newer version.

NOTE

The third principle of application layout is that code lands in source control, by default off, behind a feature flag, and is only activated through a code-reviewed change to configuration files.

Managing Your Application in Source Control

Now that we have determined that the filesystem should represent the source of truth for your cluster, the next important question is how to actually lay out the files in the filesystem. Obviously, filesystems contain hierarchical directories, and a source-control system adds concepts like tags and branches, so this section describes how to put these together to represent and manage your application.

Filesystem Layout

This section describes how to lay out an instance of your application for a single cluster. In later sections, we will describe how to parameterize this layout for multiple instances. It's worth getting this organization right when you begin. Much like modifying the layout of packages in source control, modifying your deployment configurations after the fact is a complicated and expensive refactor that you'll probably never get around to.

The first cardinality on which you want to organize your application is the semantic component or layer (for instance, *frontend* or *batch work queue*). Though early on this might seem like overkill, since a single team manages all of these components, it sets the stage for team scaling—eventually, different teams (or subteams) may be responsible for each of these components.

Thus, for an application with a frontend that uses two services, the filesystem might look like this:

```
frontend/
service-1/
service-2/
```

Within each of these directories, the configurations for each application are stored. These are the YAML files that directly represent the current state of the cluster. It's generally useful to include both the service name and the object type within the same file.

NOTE

While Kubernetes allows you to create YAML files with multiple objects in the same file, this is generally an antipattern. The only good reason to group several objects in the same file is if they are conceptually identical. When deciding what to include in a single YAML file, consider design principles similar to those for defining a class or struct. If grouping the objects together doesn't form a single concept, they probably shouldn't be in a single file.

Thus, extending our previous example, the filesystem might look like:

```
frontend/
  frontend-deployment.yaml
  frontend-service.yaml
  frontend-ingress.yaml
service-1/
  service-1-deployment.yaml
  service-1-service.yaml
  service-1-configmap.yaml
...
```

Managing Periodic Versions

What about managing releases? It is very useful to be able to look back and see what your application deployment previously looked like. Similarly, it is very useful to be able to iterate a configuration forward while still deploying a stable release configuration.

Consequently, it's handy to be able to simultaneously store and maintain multiple revisions of your configuration. There are two different approaches that you can use with the file and version control systems we've outlined here. The first is to use tags, branches, and source-control features. This is convenient because it maps to the way people manage revisions in source control, and leads to a more simplified directory structure. The other option is to clone the configuration within the filesystem and use directories for different revisions. This makes viewing the configurations simultaneously very straightforward.

These approaches have the same capabilities in terms of managing different release versions, so it is ultimately an aesthetic choice between the two. We will discuss both approaches and let you or your team decide which you prefer.

Versioning with branches and tags

When you use branches and tags to manage configuration revisions, the directory structure does not change from the example in the previous section. When you are ready for a release, you place a source-control tag (such as `git tag v1.0`) in the configuration source-control system. The tag represents the configuration used for that version, and the `HEAD` of source control continues to iterate forward.

Updating the release configuration is somewhat more complicated, but the approach models what you would do in source control. First, you commit the change to the `HEAD` of the repository. Then you create a new branch named `v1` at the `v1.0` tag. You cherry-pick the desired change onto the release branch (`git cherry-pick <edit>`), and finally, you tag this branch with the `v1.1` tag to indicate a new point release. This approach is illustrated in Figure 22-1.

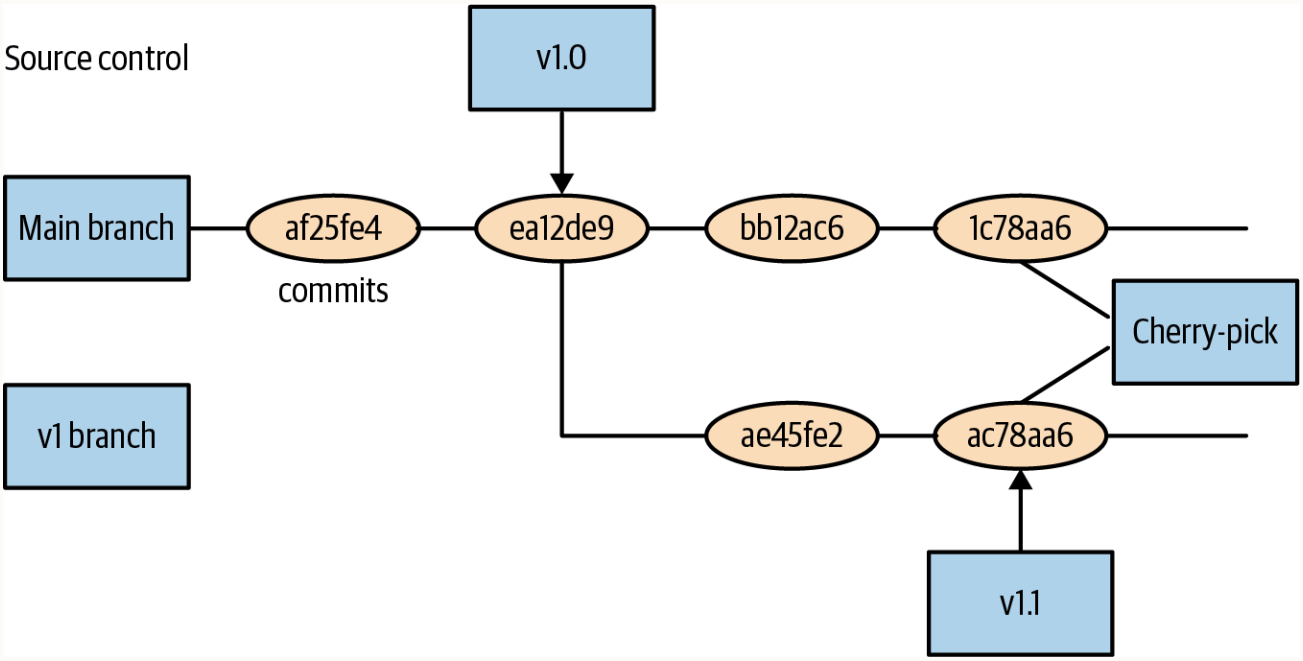


Figure 22-1. Cherry-pick workflow

NOTE

One common error when cherry-picking fixes into a release branch is to only pick the change into the latest release. It's a good idea to cherry-pick it into all active releases, in case you need to roll back versions but the fix is still needed.

Versioning with directories

An alternative to using source-control features is to use filesystem features. In this approach, each versioned deployment exists within its own directory. For example, the filesystem for your application might look like this:

```
frontend/
  v1/
    frontend-deployment.yaml
    frontend-service.yaml
  current/
    frontend-deployment.yaml
    frontend-service.yaml
service-1/
  v1/
    service-1-deployment.yaml
    service-1-service.yaml
  v2/
    service-1-deployment.yaml
    service-1-service.yaml
  current/
    service-1-deployment.yaml
    service-1-service.yaml
...
```

Thus, each revision exists in a parallel directory structure within a directory associated with the release. All deployments occur from `HEAD` instead of from specific revisions or tags. You would add a new configuration to the files in the *current* directory.

When creating a new release, you copy the *current* directory to create a new directory associated with the new release.

When you're performing a bug-fix change to a release, your pull request must modify the YAML file in all the relevant release directories. This is a slightly better experience than the cherry-picking approach described earlier, since it is clear in a single change request that all of the relevant versions are being updated with the same change, instead of requiring a cherry-pick per version.

Structuring Your Application for Development, Testing, and Deployment

In addition to structuring your application for a periodic release cadence, you also want to structure your application to enable Agile development, quality testing, and safe deployment. This allows developers to make and test changes to the distributed application rapidly and roll those changes out to customers safely.

Goals

There are two goals for your application with regard to development and testing. The first is that each developer should be able to easily develop new features for the application. In most cases, the developer is only working on a single component, yet that component is interconnected to all of the other microservices within the cluster. Thus, to facilitate development, it is essential that developers be able to work in their own environment with all services available.

The other goal is to structure your application for easy and accurate testing prior to deployment. This is essential for rolling out features quickly while maintaining high reliability.

Progression of a Release

To achieve both of these goals, it is important to relate the stages of development to the release versions described earlier. The stages of a release are:

`HEAD`

: The bleeding edge of the configuration; the latest changes.

Development

: Largely stable, but not ready for deployment. Suitable for developers to use for building features.

Staging

: The beginnings of testing, unlikely to change unless problems are found.

Canary

: The first real release to users, used to test for problems with real-world traffic and likewise give users a chance to test what is coming next.

Release

: The current production release.

Introducing a development tag

Regardless of whether you structure releases using the filesystem or version control, the right way to model the development stage is via a source-control tag. This is because

development is necessarily fast moving as it tracks stability only slightly behind `HEAD`.

To introduce a development stage, you add a new `development` tag to the source-control system and use an automated process to move this tag forward. On a periodic cadence, you'll test `HEAD` via automated integration testing. If these tests pass, you move the `development` tag forward to `HEAD`. Thus, developers can track reasonably close to the latest changes when deploying their own environments, but also be assured that the deployed configurations have at least passed a limited smoke test. This approach is illustrated in Figure 22-2.

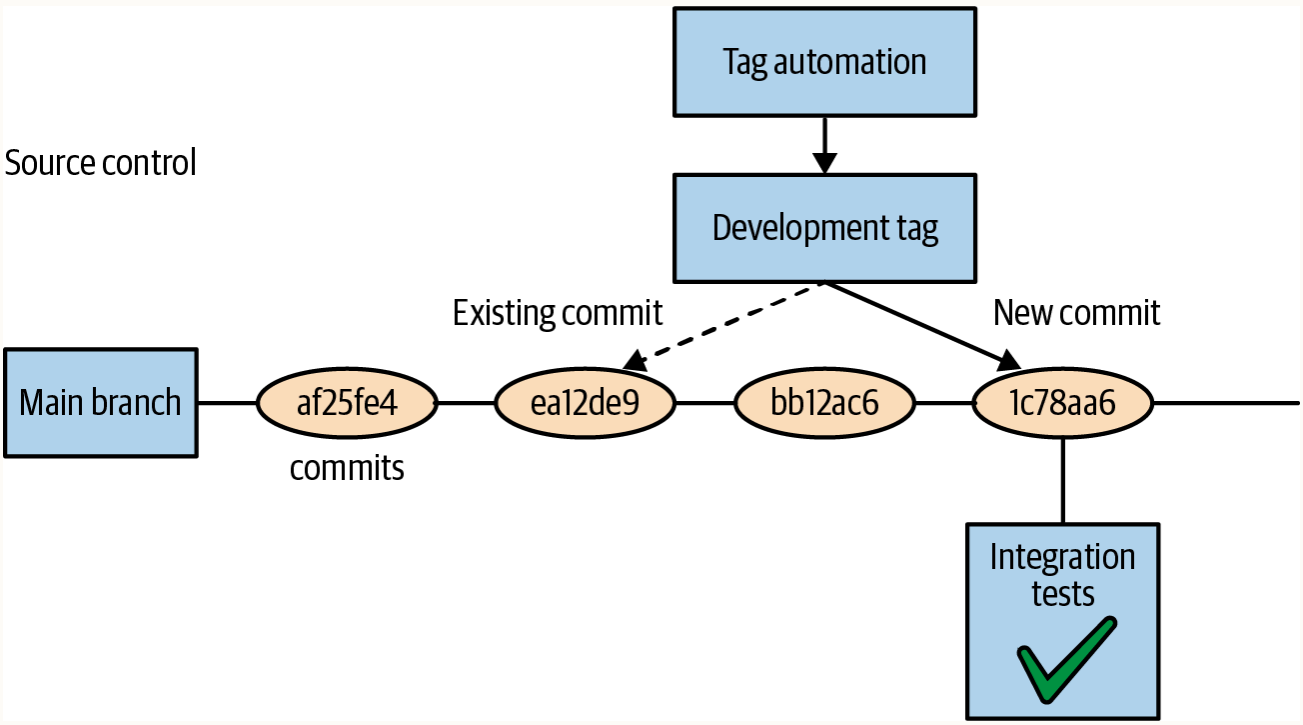


Figure 22-2. Development tag workflow

Mapping stages to revisions

It might be tempting to introduce a new set of configurations for each of these stages, but in reality, every combination of versions and stages would create a mess that would be very difficult to reason about. Instead, the right practice is to introduce a mapping between revisions and stages.

Regardless of whether you are using the filesystem or source-control revisions to represent different configuration versions, it is easy to implement a map from stage to revision. In the filesystem case, you can use symbolic links to map a stage name to a revision:

```
frontend/
  canary/ -> v2/
  release/ -> v1/
```

```
    v1/
      frontend-deployment.yaml
...
```

For version control, it is simply an additional tag at the same revision as the appropriate version.

In either case, versioning proceeds using the processes described previously, and the stages are moved forward to new versions separately as appropriate. In effect, this means that there are two simultaneous processes: the first for cutting new release versions and the second for qualifying a release version for a particular stage in the application life cycle.

Parameterizing Your Application with Templates

Once you have a Cartesian product of environments and stages, it becomes impractical or impossible to keep them all entirely identical. And yet, it is important to strive for the environments to be as identical as possible. Variance and drift between different environments produces snowflakes and systems that are hard to reason about. If your staging environment is different than your release environment, can you really trust the load tests that you ran in the staging environment to qualify a release? To ensure that your environments stay as similar as possible, it is useful to use parameterized environments. Parameterized environments use *templates* for the bulk of their configuration, but they mix in a limited set of *parameters* to produce the final configuration. In this way, most of the configuration is contained within a shared template, while the parameterization is limited in scope and maintained in a small parameters file for easy visualization of differences between environments.

Parameterizing with Helm and Templates

There are a variety of different languages for creating parameterized configurations. In general they all divide the files into a *template* file, which contains the bulk of the configuration, and a *parameters* file, which can be combined with the template to produce a complete configuration. In addition to parameters, most templating languages allow parameters to have default values if no value is specified.

The following gives examples of how to parameterize configurations using Helm, a package manager for Kubernetes. Despite what devotees of various languages may say, all parameterization languages are largely equivalent, and as with programming languages, which one you prefer is largely a matter of personal or team style. Thus, the patterns described here for Helm apply regardless of the templating language you choose.

The Helm template language uses "mustache" syntax:

```
metadata:
  name: {{ .Release.Name }}-deployment
```

This indicates that `Release.Name` should be substituted with the name of a deployment.

To pass a parameter for this value, you use a *values.yaml* file with contents like:

```
Release:
  Name: my-release
```

After parameter substitution, this results in:

```
metadata:
  name: my-release-deployment
```

Filesystem Layout for Parameterization

Now that you understand how to parameterize your configurations, how do you apply that to the filesystem layouts? Instead of treating each deployment life cycle stage as a pointer to a version, think of each deployment life cycle as the combination of a parameters file and a pointer to a specific version. For example, in a directory-based layout, it might look like this:

```
frontend/
  staging/
    templates -> ../v2
    staging-parameters.yaml
  production/
    templates -> ../v1
    production-parameters.yaml
  v1/
```

```
    frontend-deployment.yaml
    frontend-service.yaml
  v2/
    frontend-deployment.yaml
    frontend-service.yaml
...
```

Doing this with version control looks similar, except that the parameters for each life cycle stage are kept at the root of the configuration directory tree:

```
frontend/
  staging-parameters.yaml
  templates/
    frontend-deployment.YAML
...
```

Deploying Your Application Around the World

Now that you have multiple versions of your application moving through multiple stages of deployment, the final step in structuring your configurations is to deploy your application around the world. But don't think that these approaches are only for large-scale applications. You can use them to scale from two different regions to tens or hundreds around the world. In the cloud, where an entire region can fail, deploying to multiple regions (and managing that deployment) is the only way to achieve sufficient uptime for demanding users.

Architectures for Worldwide Deployment

Generally speaking, each Kubernetes cluster is intended to live in a single region and to contain a single, complete deployment of your application. Consequently, worldwide deployment of an application consists of multiple different Kubernetes clusters, each with its own application configuration. Describing how to actually build a worldwide application, especially with complex subjects like data replication, is beyond the scope of this chapter, but we will describe how to arrange the application configurations in the filesystem.

A particular region's configuration is conceptually the same as a stage in the deployment life cycle. Thus, adding multiple regions to your configuration is identical to adding new life cycle stages. For example, instead of:

- Development
- Staging
- Canary
- Production

You might have:

- Development
- Staging
- Canary
- EastUS
- WestUS
- Europe
- Asia

Modeling this in the filesystem for configuration looks like:

```
frontend/
  staging/
    templates -> ../v3/
    parameters.yaml
  eastus/
    templates -> ../v1/
    parameters.yaml
  westus/
    templates -> ../v2/
    parameters.yaml
  ...
```

If you instead are using version control and tags, the filesystem would look like:

```
frontend/
  staging-parameters.yaml
  eastus-parameters.yaml
  westus-parameters.yaml
  templates/
    frontend-deployment.yaml
...
```

Using this structure, you would introduce a new tag for each region and use the file contents at that tag to deploy to that region.

Implementing Worldwide Deployment

Now that you have configurations for each region around the world, the question becomes how to update those various regions. One of the primary goals of using multiple regions is to ensure very high reliability and uptime. While it would be tempting to assume that cloud and datacenter outages are the primary causes of downtime, the truth is that outages are generally caused by new versions of software rolling out. Because of this, the key to a highly available system is limiting the effect, or "blast radius," of any change that you might make. Thus, as you roll out a version across a variety of regions, it makes sense to move carefully from region to region, and to validate and gain confidence in one region before moving on to the next.

Rolling out software across the world generally looks more like a workflow than a single declarative update: you begin by updating the version in staging to the latest version and then proceed through all regions until it is rolled out everywhere. But how should you structure the various regions, and how long should you wait to validate between regions?

NOTE

You can use tools such as GitHub Actions to automate the deployment workflow. They provide a declarative syntax to define your workflow and are also stored in source control.

To determine the length of time between rollouts to regions, consider the "mean time to smoke" for your software. This is the time it takes on average after a new release is rolled out to a region for a problem (if it exists) to be discovered. Obviously, each problem is unique and can take a varying amount of time to make itself known, and that is why you want to understand the *average* time. Managing software at scale is a business of probability, not certainty, so you want to wait for a time that makes the probability of an error low enough that you are comfortable moving on to the next region. Something like two to three times the mean time to smoke is probably a reasonable place to start, but it is highly variable depending on your application.

To determine the order of regions, it is important to consider the characteristics of various regions. For example, you are likely to have high-traffic regions and low-traffic regions. Depending on your application, you may have features that are more popular in one geographic area than another. All of these characteristics should be considered when putting together a release schedule. You likely want to begin by rolling out to a low-traffic region. This ensures that any early problems you catch are limited to an area

of little impact. Though it is not a hard-and-fast rule, early problems are often the most severe, since they manifest quickly enough to be caught in the first region you roll out to. Thus, minimizing the impact of such problems on your customers makes sense. Next, roll out to a high-traffic region. Once you have successfully validated that your release works correctly via the low-traffic region, validate that it works correctly at scale. The only way to do this is to roll it out to a single high-traffic region. When you have successfully rolled out to both a low- and a high-traffic region, you may have confidence that your application can safely roll out everywhere. However, if there are regional variations, you may want to also test slowly across a variety of geographies before pushing your release more broadly.

When you put your release schedule together, it is important to follow it completely for every release, no matter how big or how small. Many outages have been caused by people accelerating releases, either to fix some other problem or because they believed it to be "safe."

Dashboards and Monitoring for Worldwide Deployments

It may seem an odd concept when you are developing at a small scale, but one significant problem that you will likely run into at a medium or large scale is having different versions of your application deployed to different regions. This can happen for a variety of reasons (such as, because a release has failed, been aborted, or had problems in a particular region), and if you don't track things carefully you can rapidly end up with an unmanageable snowflake of different versions deployed around the world. Furthermore, as customers inquire about fixes to bugs they are experiencing, a common question will become: "Is it deployed yet?"

Thus, it is essential to develop dashboards, which can tell you at a glance which version is running in which region, as well as alerting, which will fire when too many versions of your application are deployed. A best practice is to limit the number of active versions to no more than three: one testing, one rolling out, and one being replaced by the rollout. Any more active versions than this is asking for trouble.

Summary

This chapter provides guidance on how to manage a Kubernetes application through software versions, deployment stages, and regions around the world. It highlights the principles at the foundation of organizing your application: relying on the filesystem for

organization, using code review to ensure quality changes, and relying on feature flags, or gates, to make it easy to incrementally add and remove functionality.

As with everything, the recipes in this chapter should be taken as inspiration, rather than absolute truth. Read the guidance, and find the mix of approaches that works best for the particular circumstances of your application. But keep in mind that in laying out your application for deployment, you are setting a process that you will likely have to live with for years.

Previous chapter
< 21. Multicluster Application Deployments

Next chapter
A. Building Your Own Kubernetes Cluster >